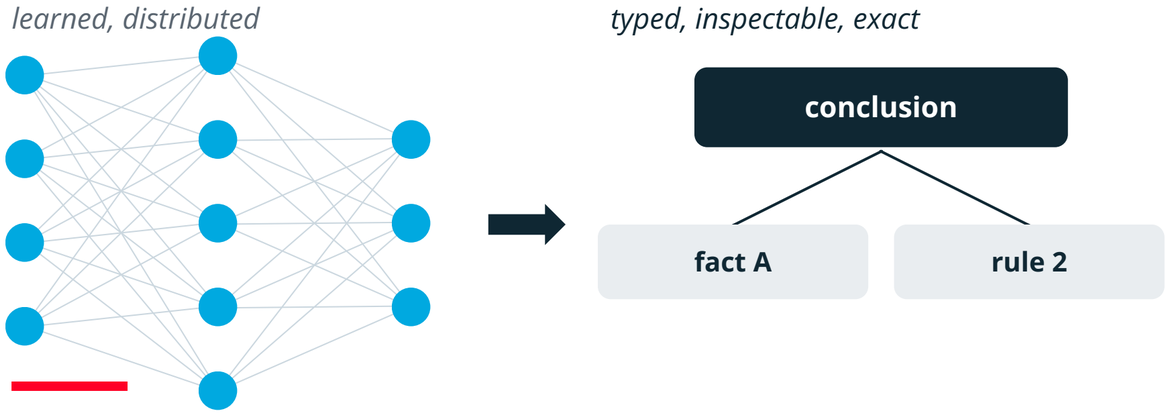
Neurosymbolic AI

Neural learning meets symbolic reasoning



3 hours · 3 blocks · 2 optional labs

Jan Polišenský
ipolisensky@fit.vut.cz
Brno University of Technology
Faculty of Information Technology

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

What to expect

Jan Polišenský

- Researcher at BUT FIT: network security, DNS threat detection
- Cloud and ML engineering, MLOps
- Co-founder and CTO of Lakmoos AI
- Trainer and consultant on applied AI

1. **Two traditions**
   Why neural alone is not enough
2. **Core ideas**
   Taxonomy, building blocks, architectures
3. **In practice**
   Applications, limits, discussion

+ two short tasks you run yourself

Interrupt at any point. The examples matter more than the definitions.

Learning objectives

What you should take away from this hour

1. **Core concepts**: what makes a system neurosymbolic rather than merely hybrid
2. **Key architectures**: who is in control, and where the interface sits
3. **Interpretability**: what structure buys you, and what it costs
4. **Use cases**: where the approach is already deployed
5. **Open problems**: what the field has not solved

By the end you should be able to look at a system and say which parts are learned, which are computed, and what is actually guaranteed.

How the session runs

Three blocks, two breaks, two optional labs

Block 1	BREAK	Block 2	BREAK	Block 3
Two traditions, logic and inference	10 min	Core concepts, logic in the network	10 min	Solvers, practice, evaluation
55 min		*55 min*		*50 min*

Two hands-on labs sit inside blocks 2 and 3. They are extensions, run if the pace allows.

Blocks 1 and 2

The two traditions, then what a neurosymbolic system is, and how logic gets inside a network.

Block 3

Solvers, planners, knowledge graphs, and how to read the literature critically.

Optional labs

Bring a laptop with Python 3.9+ if you have one.
No installs, no internet, no API key.

Interrupt whenever. Three hours is long enough that a question is a favour to everyone in the room.

01

Two traditions

Why neural alone is not enough

Fluent is not the same as correct

A pendulum that has swung twice

Two failure modes, exactly opposite

Warm-up: which of these would you sign off?

Thirty seconds, no discussion yet

A **A model reads a contract and reports that clause 7 caps liability at €2m.**

It is right 94% of the time.

B **A model computes a drug dose and explains its reasoning step by step.**

The explanation is coherent and correct.

C **A model writes an SQL query, the database runs it, you get a number.**

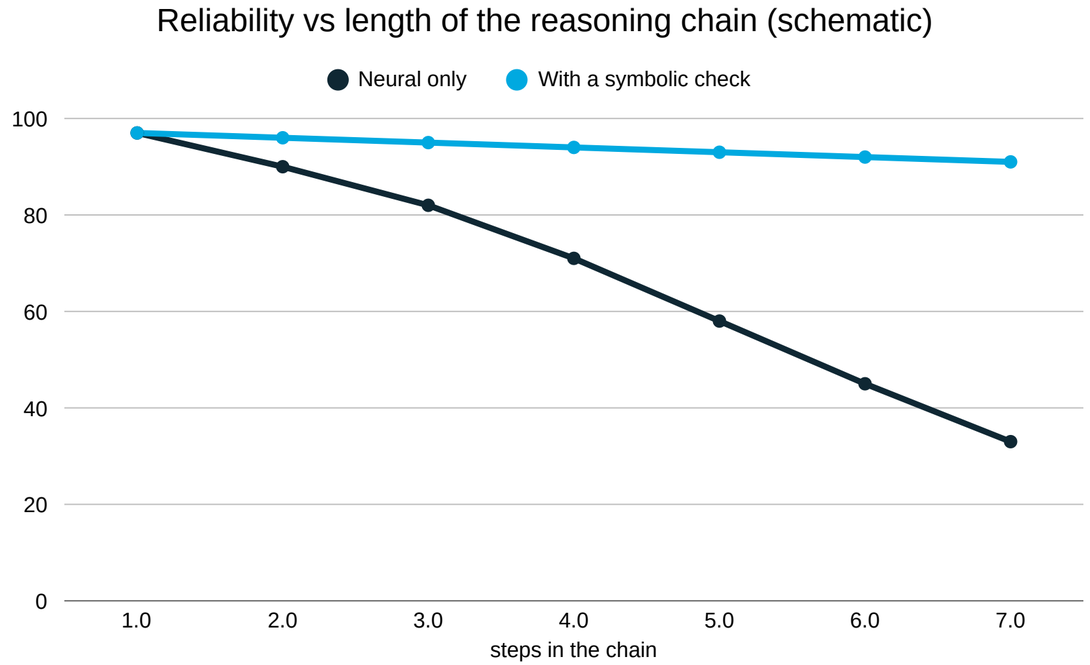
The query executed without error.

Hands up for each. Hold the answers. We come back to all three.

Why the question comes up

Fluent is not the same as correct

- A model returns the most probable continuation, not a verified conclusion
- The same prompt sampled twice can give two answers
- The explanation is written after the fact and need not describe the real computation
- Hard constraints, budgets, dosages, legal limits, can be violated silently
- Accuracy decays with the length of the reasoning chain

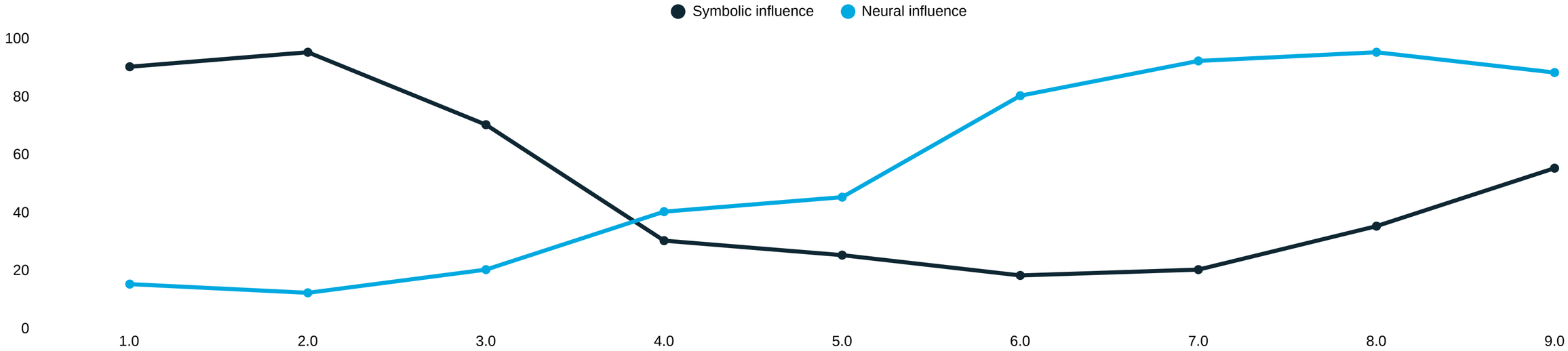


Schematic, not measured. The shape is the point: independent steps multiply.

None of this is a reason to abandon neural models. It is a reason not to let them decide everything.

The pendulum has swung twice

Neither tradition disappeared. One of them stopped being called AI.



schematic. The shape is the argument, not the numbers.

1956–85
The symbolic era
Logic Theorist, DENDRAL, MYCIN, Prolog

1985–95
The AI winter
Expert systems broke in the field. Funding fell

1986–2012
Connectionism returns
Backpropagation, then ImageNet

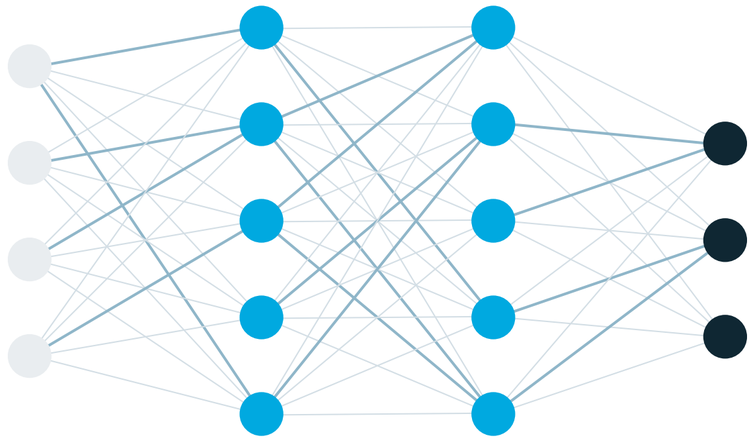
2012–22
Deep learning wins
Perception is solved. Exactness is not

2022–
The hybrid revival
Rediscovering structure, proof, guarantees

The Lighthill report (1973) attacked combinatorial explosion, not hardware. Systems worked on the cases their authors had imagined.

The neural tradition

Learn a function from data. Knowledge lives in the weights.



Where is “cat” stored?

Spread across the edges. Delete any single weight and the model still classifies cats; there is no single weight to point at.

Strong at	Weak at
Perception: vision, speech, signals	Exactness and long chains
Language and similarity	Guarantees, constraints
Noisy, incomplete input	Systematic compositionality
Learning instead of specifying	Tracing one decision

Distribution is why it generalises and why it cannot be audited. One property, both times.

A question to carry through the rest of the session: could you print the reason for this answer?

Two ways to store the same fact

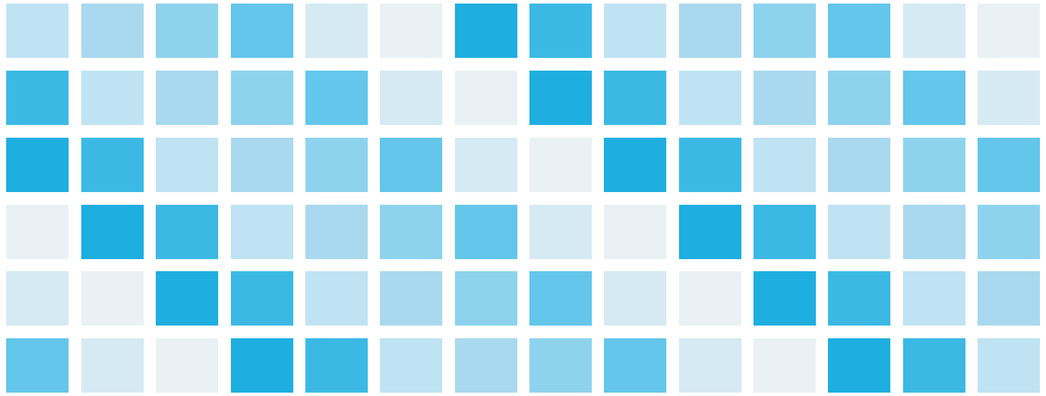
One has an address, the other has a pattern

Localist (symbolic)

entity	property	value
cat	**is_a**	**mammal**
cat	has	fur
dog	is_a	mammal
car	has	wheels

One row. Delete it and the fact is gone. Edit it and nothing else moves.

Distributed (neural)



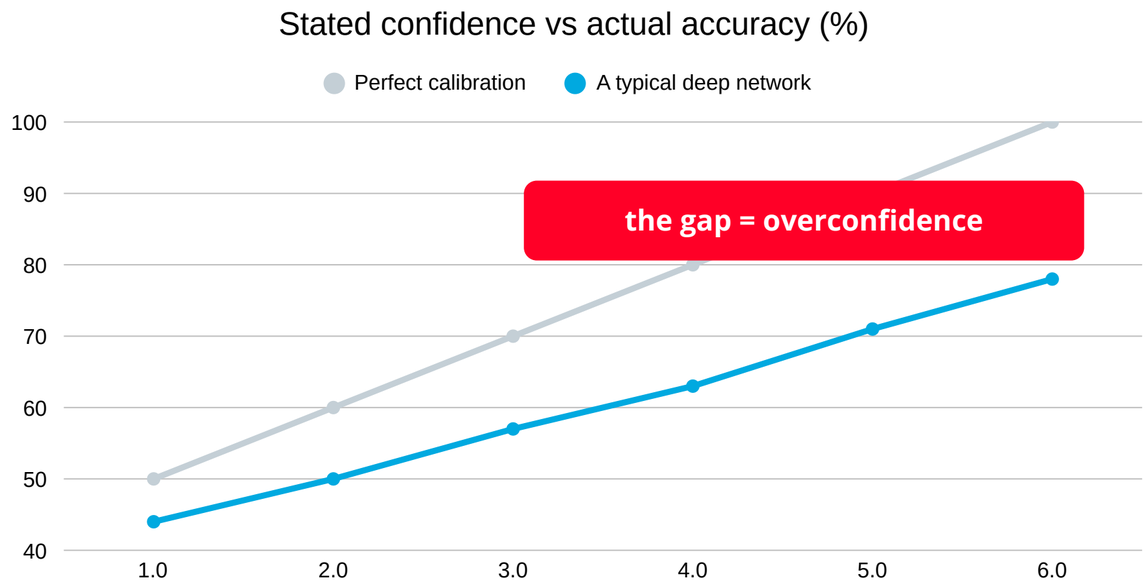
w_1 … w_{84}

No cell means “cat”. The fact is the whole pattern, and the same cells also carry “dog” and “car”.

Edit one fact	one row	retrain, and everything shifts
Explain an answer	cite the row	no address to cite
Generalise to “lynx”	add a row by hand	free: similar things sit nearby

Why 0.97 is not 97% confident

Softmax normalises scores; it does not measure correctness



Schematic, after Guo et al., 2017. Shapes vary by model; the direction does not.

1. **What softmax does**
   Turns arbitrary scores into numbers that sum to 1. Normalisation, not measurement.

2. **What it is trained for**
   Minimising loss. Nothing in that objective asks the number to be honest.

3. **What actually happens**
   Deeper, more accurate networks became worse calibrated, not better.

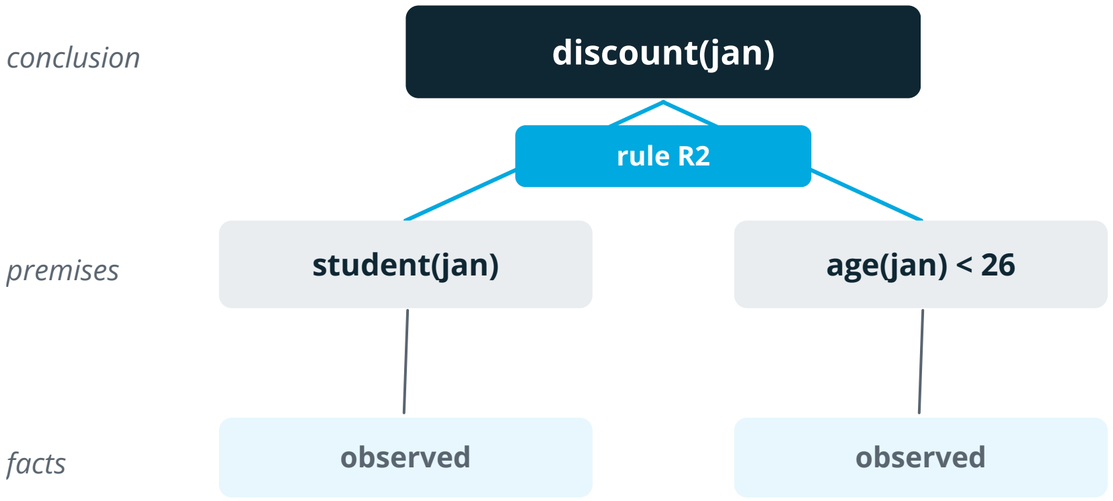
4. **What survives**
   Ranking. 0.97 still usually beats 0.62, so tuned thresholds work.

5. **What does not**
   The reading “wrong three times in a hundred”. That claim is unsupported.

Lab 2 is built on this gap: extraction errors arrive with high confidence attached.

The symbolic tradition

Every node has a name, every edge is a rule



This picture is the explanation. Nothing was generated to produce it. It is the computation, printed.

Strong at	Weak at
Exact, repeatable results	Anything outside the model
Auditable derivations	Acquiring the knowledge
Verification, hard guarantees	Raw perception, messy language
Data efficiency and reuse	Combinatorial search cost

Prolog · MYCIN · knowledge graphs · SAT and SMT solvers · PDDL planners

Someone wrote rule R2 and someone has to keep it current as the policy changes.

Logic in five minutes

Enough notation to read the rest of the slides

Propositional logic

Whole statements, true or false, combined with connectives.

`raining ∧ no_umbrella → gets_wet`

∧ and ∨ or ¬ not → implies

Limit: "raining" is atomic. You cannot say who is getting wet.

First-order logic

Objects, predicates about them, and quantifiers.

`∀x student(x) ∧ age(x) < 26 → discount(x)`

∀ for all ∃ there exists x a variable

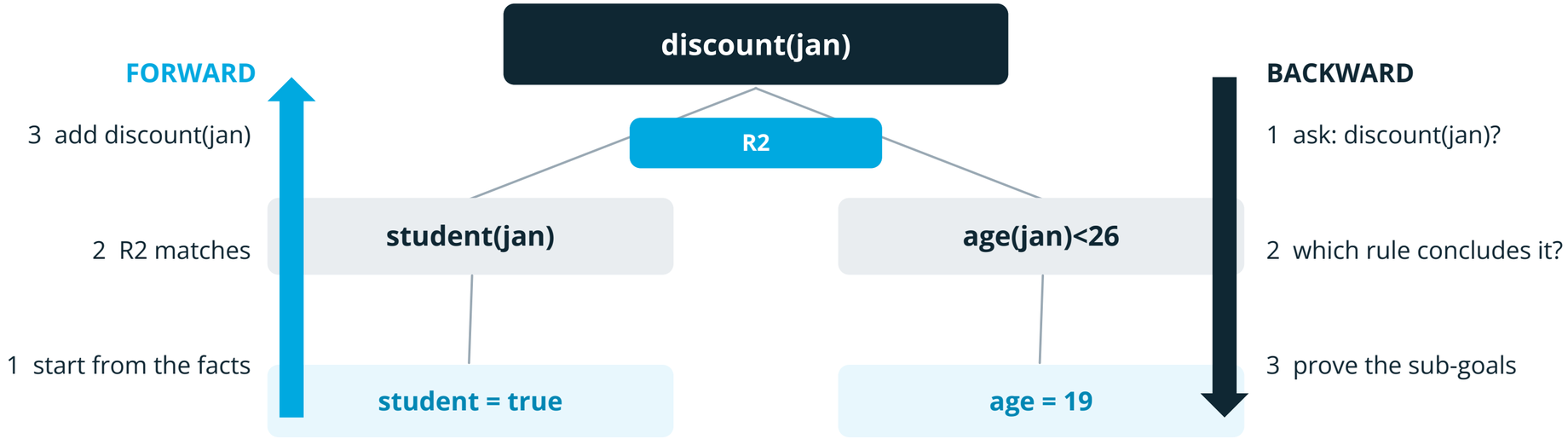
One rule now covers every passenger who ever exists.

Predicate	student(jan): true or false of a given object
Variable	x: a placeholder a rule can bind to any object
Ground fact	a statement with no variables left. age(jan, 19)
Entailment	"follows from". If the premises hold, the conclusion must

That is the whole vocabulary. Everything later in this lecture is these four words plus engineering.

Two directions through the same rules

Forward chains from what you know. Backward chains from what you asked.



data-driven: derive everything that follows. Monitoring, alerting.

goal-driven: touch only what the question needs. Prolog, diagnosis.

Work done	proportional to everything derivable	proportional to what the goal needs
Good when	few facts, many consequences	many rules, one question
In the wild	rules engines, RETE (Forgy, 1982)	Prolog, SLD resolution

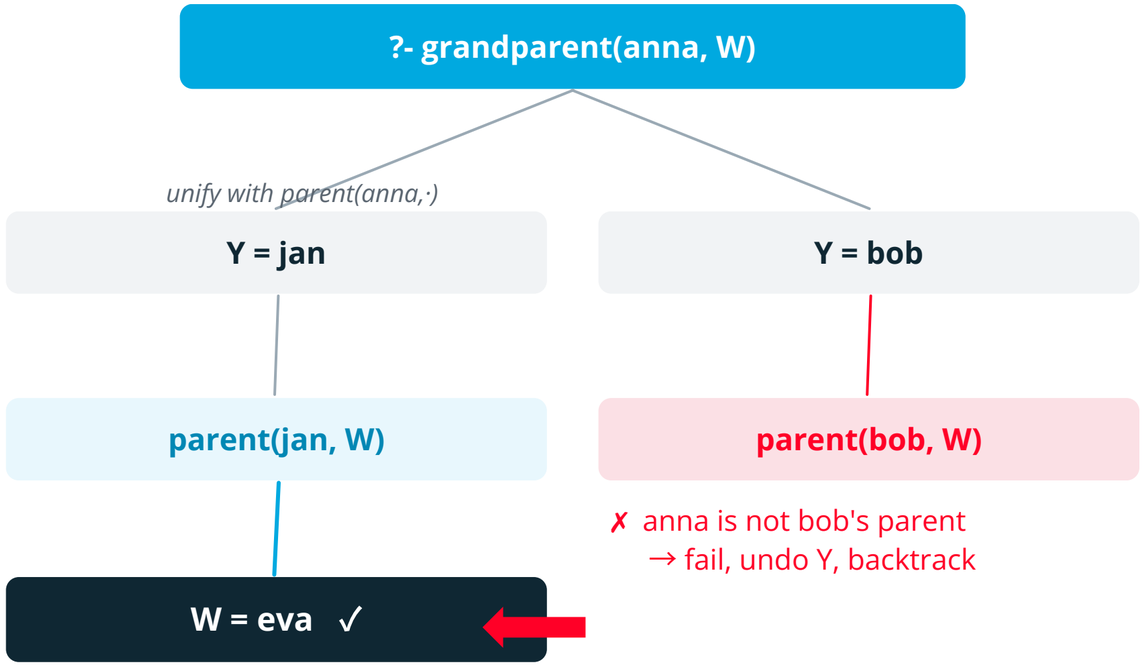
Prolog: the search comes with the language

Unification matches goals to rules; backtracking undoes failed bindings

Program

```
parent(anna, jan).
parent(jan,  eva).
parent(bob,  ida).

grandparent(X, Z) :-
    parent(X, Y),
    parent(Y, Z).
```

:-	means “if”. Read the rule right to left.
Capitals	are variables; lowercase are constants
Unification	match a goal to a rule head, bind variables (Robinson, 1965)
Backtracking	a branch fails → undo the bindings, try the next clause



You wrote what is true. The search strategy came free and so did the record of which branch failed.

Rule engines in practice

What goes wrong once you have more than ten rules

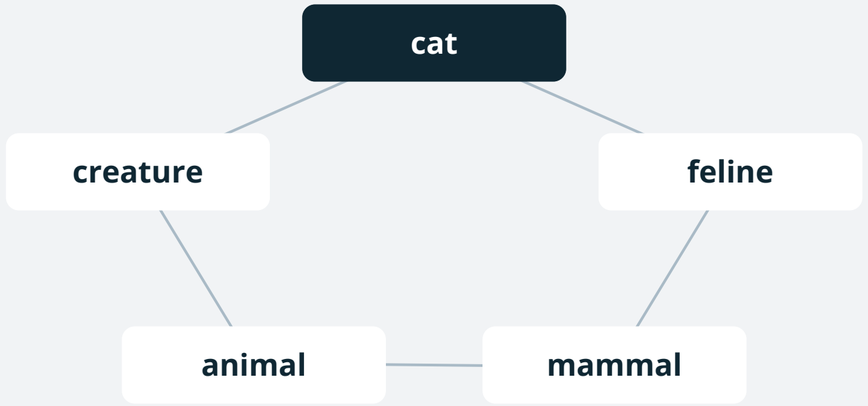
Conflict	Two rules match the same facts and disagree.	Order them. First match wins, or explicit priorities.
Precedence	A specific rule and a general one both apply.	Specific beats general, but write it down. Do not rely on file order.
Coverage gaps	Nothing matches and the engine returns silence.	A total fallback. Silence is not a decision.
Contradiction	The rule set entails both P and ¬P.	From a contradiction, classical logic entails everything. Test for it.
Drift	The policy changed and the rules did not.	Rules are code: version control, review, tests, an owner.

Two of these are in Task 1: rule order decides the 14-year-old with a student card, and the fallback decides everyone else.

The symbol grounding problem

Harnad, 1990. The objection the symbolic tradition never answered.

A dictionary with no pictures



Every definition leads to another symbol. The loop never touches the world.

What the neural half added

The symbol is now anchored in something that is not another symbol.

… and it is now something that can be measured, and be wrong.

Why it mattered	the strongest argument that a purely symbolic system computes rather than understands
The neighbouring puzzle	the frame problem (McCarthy & Hayes, 1969): stating what an action leaves unchanged
What we actually did	we turned a philosophical problem into an engineering component with an error rate

Each tradition's hardest problem is the other one's strength.

Side by side

The failure modes are complementary

	Neural	Symbolic
Knowledge	Weights	Facts and rules
Acquired by	Training	Authoring
Explanation	Post-hoc	The derivation
Typical failure	**Confidently wrong**	**No answer**
Data required	Large	Small
Guarantees	Statistical	Logical

Which of those two failures would you rather be paged about at three in the morning?

Block 1 in six lines

Before the break. If any of these is unclear, ask now.

1. Neural systems learn a function from data; knowledge lives distributed across weights
2. Symbolic systems store facts and rules explicitly and derive conclusions by inference
3. A softmax score is a ranking, not a probability of being correct
4. First-order logic buys you one rule that covers every object, forever
5. Inference runs forwards from facts or backwards from a goal. Same rules, different work.
6. The two traditions fail in opposite ways: confidently wrong, versus no answer at all

After the break we stop comparing them and start joining them.

BREAK

10 minutes

Just finished: the two traditions and why they fail differently

Coming up: what a neurosymbolic system actually is

02

Core concepts

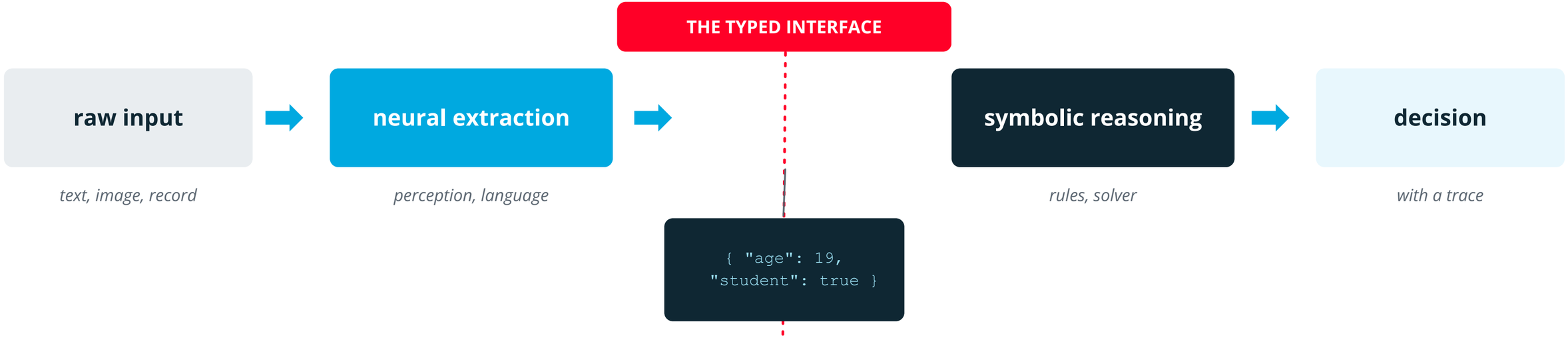
Taxonomy, building blocks, architectures

- What makes a system neurosymbolic
- Kautz's six combinations
- Grounding: the weakest link

What neurosymbolic AI is

Two halves, and a defined interface between them

A neurosymbolic system uses neural components to turn messy input into symbols, and symbolic components to reason over those symbols under explicit rules.



THE TEST

Delete the whole left half. Hand-write the JSON yourself. Does the right half still run and give the same answer?
If yes, it is neurosymbolic. If no, the line does not exist and you have one model with a prompt that mentions rules.

In lab 1 you run exactly this test: the reasoner you write has no model anywhere in it.

Kautz's taxonomy

Six shapes, differing in which component is in control

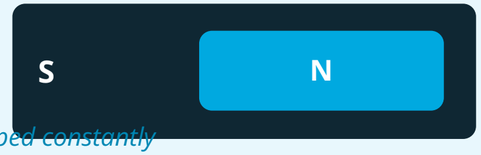
1. Symbolic Neuro Symbolic

Symbols in, symbols out, a network in the middle. Standard NLP.

shipped constantly

2. Symbolic[Neuro]

A symbolic engine calls a neural subroutine. AlphaGo: search in control.



shipped constantly

3. Neuro | Symbolic

A pipeline. The network produces structure, the reasoner consumes it.

shipped constantly

4. Neuro : Symbolic → Neuro

Symbolic knowledge compiled into training data or supervision.

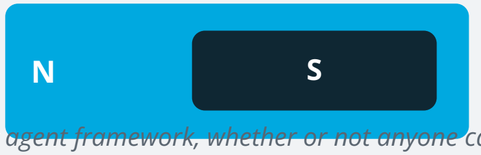
compiled into

5. Neuro_Symbolic

Logic fused into the loss or the architecture. Logic Tensor Networks.

6. Neuro[Symbolic]

The network calls a symbolic engine as a tool. An LLM plus a solver.



every agent framework, whether or not anyone calls it this

Nesting shows control. The outer component decides, the inner one is called.

Classify these four

One minute in pairs. Name the type, then say what is guaranteed.

A A chess engine that evaluates positions with a neural network and searches with alpha-beta.

B A model that reads an invoice and fills a JSON schema, which a payment rules engine then checks.

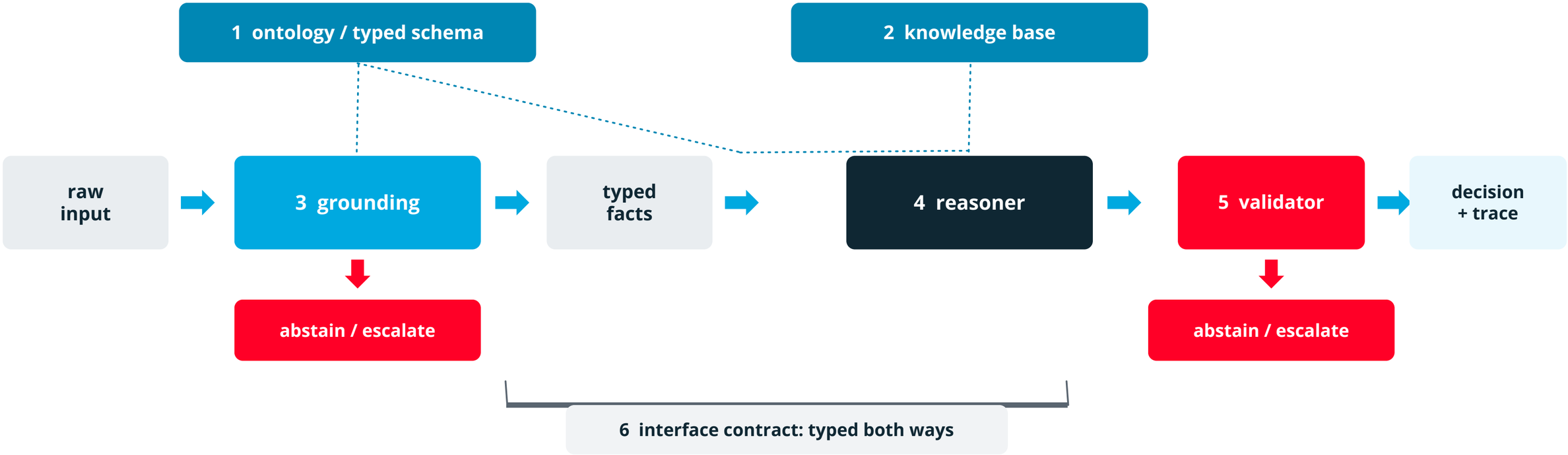
C A network trained with an extra loss term for every output that breaks a physical law.

D An assistant that writes Python, runs it in a sandbox, and reports the number it printed.

Second question, harder than the first: in which of the four could a wrong answer arrive with a perfectly convincing explanation?

Building blocks

One architecture. Six named parts, and two that students always forget.



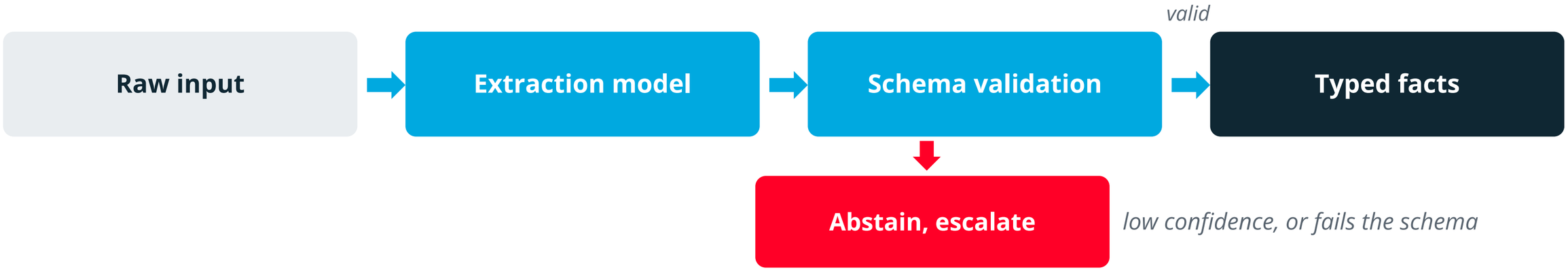
1 Ontology / schema	which facts exist, allowed values, units, ranges
3 Grounding	raw input onto symbols. Usually the weakest link.
5 Validator	type and coverage checks, plus a defined fallback

2 Knowledge base	the facts, the rules, the constraints, their precedence
4 Reasoner	rule engine, logic program, constraint or SMT solver, planner
6 Interface contract	typed data in both directions, and who wins on conflict

the two in red are usually built last, and they decide whether the system fails loudly or quietly

Grounding: raw input into symbols

The weakest link in the chain



- **Wrong facts in**: auditable nonsense out
- **Typical errors**: mis-extraction, wrong units, a missing value read as false, values outside the schema
- **Mitigations**: validate against the schema, keep a confidence score, abstain instead of guessing
- **Test it alone**: grounding needs its own labelled set, measured separately from the rules

A perfectly correct rule engine over incorrect facts still produces a confident, fully traceable wrong answer.

The guarantee spectrum

“Constrained” means four different things. Ask which one.

soft: encouraged — *hard: enforced*

Prompted

“Please stay under €2m.”

What holds:
Nothing. The wording is a preference.

Trained

Semantic loss, LTN, DeepProbLog.

What holds:
The violation rate drops. It does not reach zero.

Decoded

Grammar or automaton at generation time.

What holds:
Output is in the language. Exactly, by construction.

Checked

Post-hoc validator or solver, with a refusal path.

What holds:
Nothing leaves the system unchecked.

Only the right-hand pair are guarantees. The other two reduce a rate.

Getting typed facts out of a model

Four ways, increasing in strength

1 **Ask nicely**	`"Reply in JSON."`	Usually works. Sometimes returns prose, a code fence, or valid JSON with invented fields.	**no guarantee**
2 **Ask and retry**	`Parse; on failure, ask again.`	Converges in practice. Costs latency, and a persistent failure loops forever unless you cap it.	**eventually**
3 **Function / tool schema**	`Provider validates against your schema.`	Strong in practice. You are trusting the provider's implementation, not a proof.	**usually exact**
4 **Constrained decoding**	`Grammar restricts the tokens at each step.`	Exact by construction. Invalid output is unreachable, not merely improbable.	**exact**

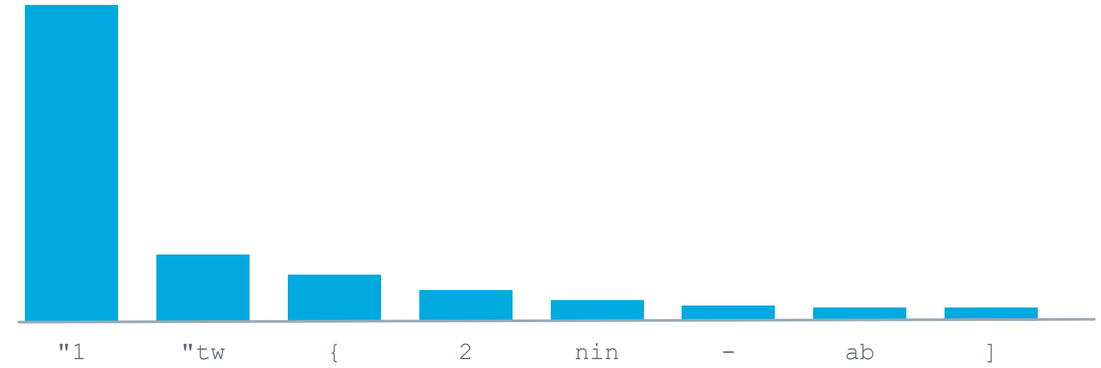
All four give you well-formed facts. None gives you true facts. That is lab 2.

Constrained decoding, mechanically

Illegal tokens are removed from the distribution, not down-weighted

```
emitted so far:   { "age":          →   what may legally come next?
```

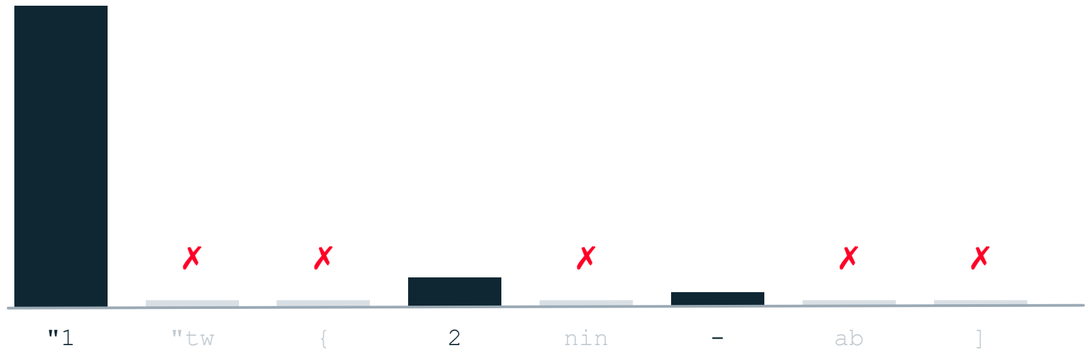
1. the model's distribution



every token has some probability, including the impossible ones

grammar masks

2. after masking and renormalising



probability zero: unreachable, not just unlikely

What you get	output provably in the language: valid JSON, valid SQL, a well-typed term
What you do not get	a true value. { "age": 2 } is perfect JSON and the wrong fare
Cost	you need a grammar, and masking adds work at every single step
Where you meet it	JSON mode, GBNF grammars, Outlines, guidance, structured-output APIs

Constraining the form is cheap and exact. Constraining the meaning is neither.

Optional lab 1: schema first, rules second

If the pace allows. Laptops open, pairs are fine.

What you do

1. **Read schema.json**: the typed interface. It is the contract, and it is deliberately the first file.
2. **Implement decide(facts)**: four ordered rules, first match wins, plus the defined fallback.
3. **Return a trace**: which rule fired, and which facts that rule actually read.
4. **Refuse bad input**: missing, mistyped or out-of-range facts must abstain, never guess.

```
python3 check_task1.py      →      16 / 16 cases
```

Get the files

Python 3.9+. No installs, no internet, no API key.

Before you stop: decide() is a pure function. Name two properties that follow from that.

2b

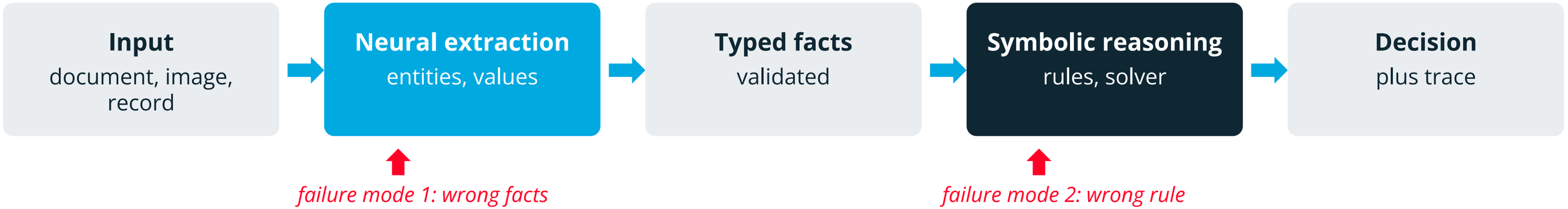
Architectures

Part 2, continued

- Neural to symbolic: the pipeline
- Knowledge inside the network
- The network calls the solver

Pattern 1: neural to symbolic

Kautz type 3: the pipeline, and the most common shape in industry.



- **The neural stage never decides**: it only describes what it sees
- **The reasoning stage is deterministic**: the same facts give the same conclusion
- **Errors localise**: a wrong answer is either a bad extraction or a wrong rule, and you can tell which
- **Typical use**: compliance checks, document understanding, eligibility and pricing
- **Cost**: the schema and the rules have to exist and someone has to maintain them

This is the pattern built in labs 1 and 2, with the two failure modes measured separately.

Pattern 2: knowledge inside the network

Kautz type 5. Everything here is soft except one.

soft: the violation rate falls — *hard: violation is impossible*

Semantic loss
penalise outputs that break a constraint during training
Xu et al., 2018

Logic Tensor Networks
first-order formulas relaxed into differentiable operations
Badreddine et al., 2022

DeepProbLog
probabilistic logic programs with neural predicates
Manhaeve et al., 2018

KG embeddings
symbolic structure represented as geometry
many

Constrained decoding
generation restricted to a grammar at inference time
the exception

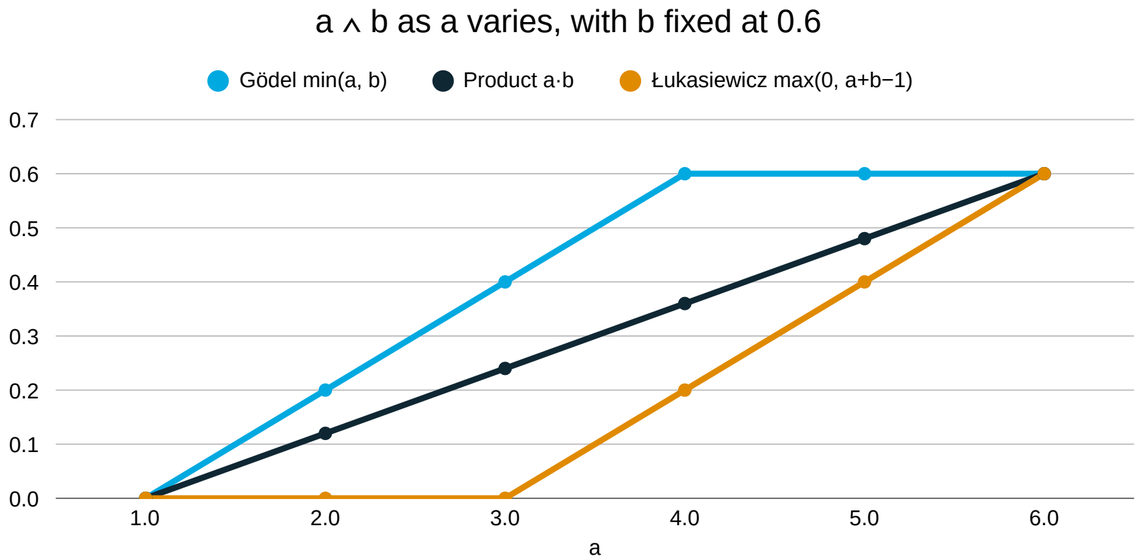
Why soft is still worth it	It shapes the whole output distribution, including cases your validator never thought to enumerate.
Why soft is not a guarantee	After training the constraint is satisfied often. “Often” is a measurement, not a promise.
Why decoding is different	It removes the violating token from the distribution. Sampling cannot reach it.

Four of these lower the violation rate. One makes violation impossible.

Making logic differentiable

You cannot backpropagate through true and false. So relax them.

Replace {true, false} with the interval [0,1]. A t-norm is any function that behaves like AND on that interval and agrees with Boolean logic at the corners.



flat stretch = zero gradient. That is why the choice is an engineering decision.

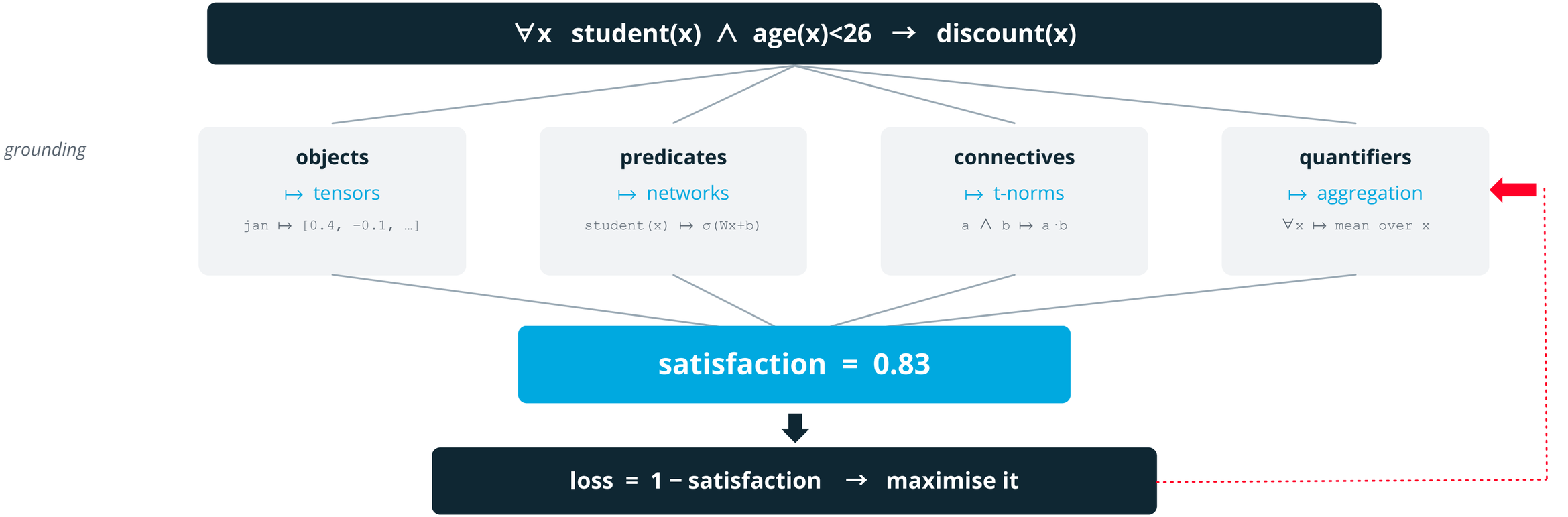
¬ a	`1 − a`	*negation*
a ∨ b	`a + b − a·b`	*the dual t-conorm*
a → b	`1 − a + a·b`	*Reichenbach implication*
∀x P(x)	`mean over x`	*aggregate, not conjoin*
∃x P(x)	`soft maximum over x`	*aggregate, not disjoin*

Sanity check: at a = 1 and b = 1 all three give 1. At a = 0 all three give 0. They agree with Boolean logic at the corners and interpolate in between.

A rule can now be 0.83 satisfied. 0.83 satisfied is not satisfied.

Logic Tensor Networks

Badreddine et al., 2022. A theory becomes a loss function.



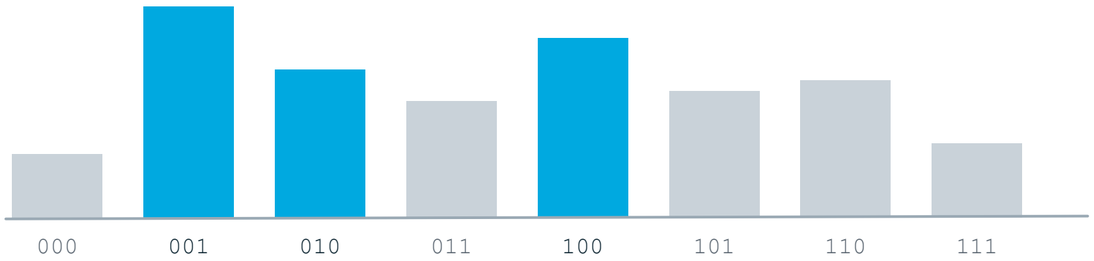
What it buys	you can state background knowledge you have no labels for, and it shapes learning
What it does not	enforcement. High satisfaction is a measurement, not a proof

Semantic loss

Xu et al., ICML 2018. Push probability mass onto the legal configurations.

constraint: exactly one of the three outputs may be true

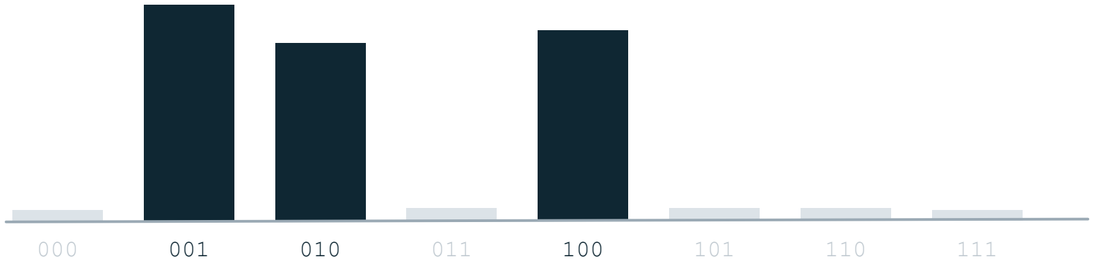
before: mass is spread over impossible worlds too



blue = satisfies the constraint grey = impossible

train

after: the constraint did the supervising



no labels were used, only the rule

$$L = -\log \sum_{\text{over legal configurations}} \prod p_i \cdot \prod (1-p_j)$$

Why it is expensive	that sum is weighted model counting, #P-hard in general
How they make it work	compile the constraint once into a circuit, then evaluate cheaply forever
What it still is	a penalty. The network violates less often, not never

Probabilistic logic: possible worlds

How ProbLog gets a number out of a proof. This is what DeepProbLog runs on.

A probabilistic program

```
0.6 :: rain.
0.3 :: sprinkler.

wet :- rain.
wet :- sprinkler.

?- wet.
```

Every combination is a world

rain, sprinkler	0.18	**wet**
rain, no sprinkler	0.42	**wet**
no rain, sprinkler	0.12	**wet**
no rain, no sprinkler	0.28	dry

P(wet) = 0.18 + 0.42 + 0.12 = 0.72

- **The semantics** — the probability of a query is the total probability of the worlds in which it is true (Sato, 1995)
- **The catch** — there are 2^n worlds. Enumerating them is hopeless past a few dozen facts
- **The fix** — compile the proofs into a circuit once, then evaluate it. Same trick as semantic loss.
- **Why you care** — swap a fixed probability for a neural network's output and you have DeepProbLog

Logic says whether. Probability says how often. This is how you get both from one program.

DeepProbLog and the MNIST addition trick

Manhaeve et al., NeurIPS 2018

You are given two handwritten digit images and their sum. You are never told what either digit is. Learn to read digits.

The whole program

```
nn(m_digit, [X], Y, [0..9]) :: digit(X, Y).

addition(X, Y, Z) :-
    digit(X, A),
    digit(Y, B),
    Z is A + B.
```

1. **digit/2 is a neural predicate**
2. **The rule is exact arithmetic**
3. **Supervision arrives at the sum**
4. **Gradient flows back through the logic**

The symbolic layer converts weak supervision into strong supervision. Knowing 3 + 5 = 8 constrains what the two images can be.

Why it matters

A label of 0 forces both digits to 0; a label of 18 forces both to 9. Every label in between removes most of the 100 possible pairs. The arithmetic is never learned, so it never breaks.

Cost: inference sums over possible worlds, and that gets expensive fast.

Kautz type 5. The rule supplies supervision the labels never contained.

Symbols as geometry

Knowledge-graph embeddings: symbols compiled into geometry

Give every entity and every relation a vector, and choose the vectors so that head + relation ≈ tail.

TransE, Bordes et al., 2013

```
v(Prague) + v(capital_of)  ≈  v(Czechia)
v(Vienna) + v(capital_of)  ≈  v(Austria)

So:  v(Berlin) + v(capital_of)  =  ?
```

What it buys	You can score a triple nobody ever stored. Link prediction over an incomplete graph.
What it costs	The answer is a nearest neighbour, not an entailment. It has no proof and no guarantee.
Kautz type	4: symbolic knowledge compiled into a form the network can use.
The family	TransE, DistMult, ComplEx, RotatE. They differ in which relation patterns they can represent.

Everywhere else in this lecture the model produces symbols. Here the symbols produce a model.

Reasoning becomes vector arithmetic: fast, approximate, and unable to report uncertainty.

Useful when the graph is incomplete and you would rather have a ranked guess than no answer at all.

Block 2 in six lines

Second checkpoint. Last easy moment to ask.

1. A system is neurosymbolic when the two halves are separable at a typed interface
2. Kautz's categories differ in one thing: who is in control, and who is the subroutine
3. Grounding (raw input into symbols) is the weakest link, and its failures are quiet
4. “Constrained” means four different things; only decoding and checking are guarantees
5. t-norms make logic differentiable, at the price of constraints that only nearly hold
6. DeepProbLog: the rule supplies supervision the labels never contained

After the break: the engines themselves, then what any of this is worth in practice.

BREAK

10 minutes

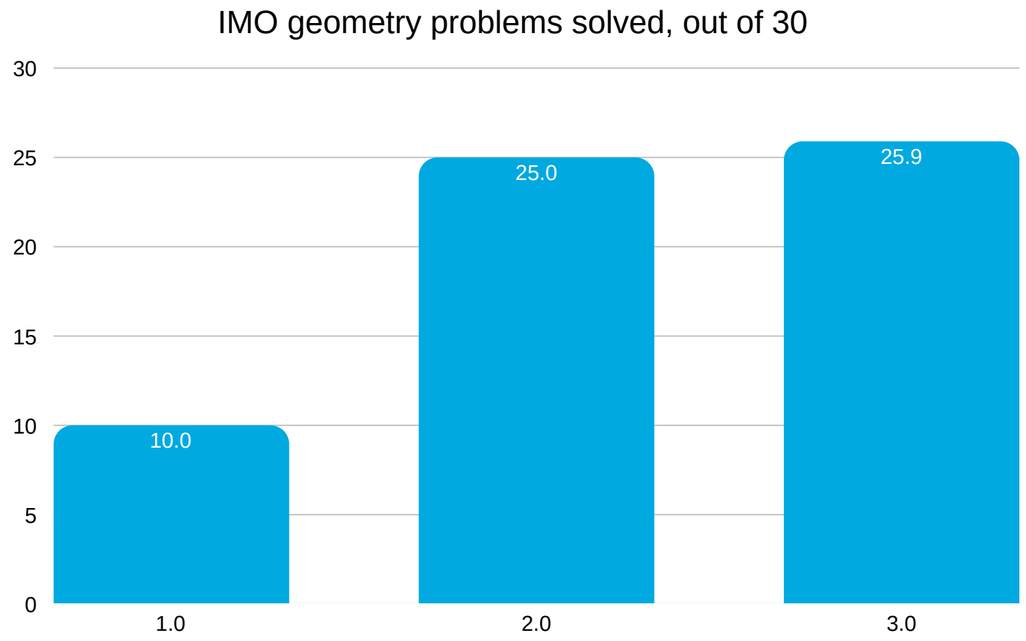
Just finished: the methods that put logic inside the network

Coming up: solvers, planners, and how to tell good work from hype

Pattern 3: the network calls the solver

Kautz type 6: the network proposes, the engine checks

- The model writes a program, a query or a formula; an external engine runs it
- In practice: Python, SQL, Prolog, an SMT solver such as Z3, a computer algebra system
- AlphaGeometry (Trinh et al., Nature 2024): the model proposes auxiliary constructions, a symbolic engine builds the proof
- Why it works: generating a candidate is hard, checking one is cheap
- The risk: a well-formed query that answers the wrong question



Source: Trinh et al., Nature 2024

Generate-and-check works when checking is cheaper than generating.

What a solver actually does

SAT, then SMT, in one slide

SAT

Given a Boolean formula, is there an assignment making it true?

$(a \lor \neg b) \land (b \lor c) \land (\neg a \lor \neg c)$

- Unit propagation, a clause with one option left forces it
- Clause learning, on conflict, record why, never repeat it
- Backjumping, undo many decisions at once, not one

NP-complete (Cook, 1971). Solvers still close industrial problems with millions of clauses.

SMT

SAT, plus theories: arithmetic, arrays, bit-vectors, uninterpreted functions.

$x > 0 \land y = 2 \cdot x \land y < 10$

The SAT engine handles the structure; a theory solver decides whether the arithmetic is consistent. Answer: $x \in \{1,2,3,4\}$.

Z3 (de Moura and Bjørner, 2008) is the one you will meet. It returns a satisfying model, or a proof that none exists.

It is exact	no sampling, no threshold, no temperature
It is total	SAT, UNSAT, or timeout, and it says which
It explains itself	a satisfying model, or an unsat core naming the conflicting constraints
It is the perfect partner	a model proposes, the solver disposes

“Timeout” is a third answer. The neural half has no equivalent.

Planning: PDDL

Where perception meets a plan that can be checked before it runs

An action, fully specified

```
(:action pick-up
  :parameters (?b - block)
  :precondition (and (clear ?b)
                     (on-table ?b)
                     (arm-empty))
  :effect (and (holding ?b)
               (not (arm-empty))
               (not (on-table ?b))))
```

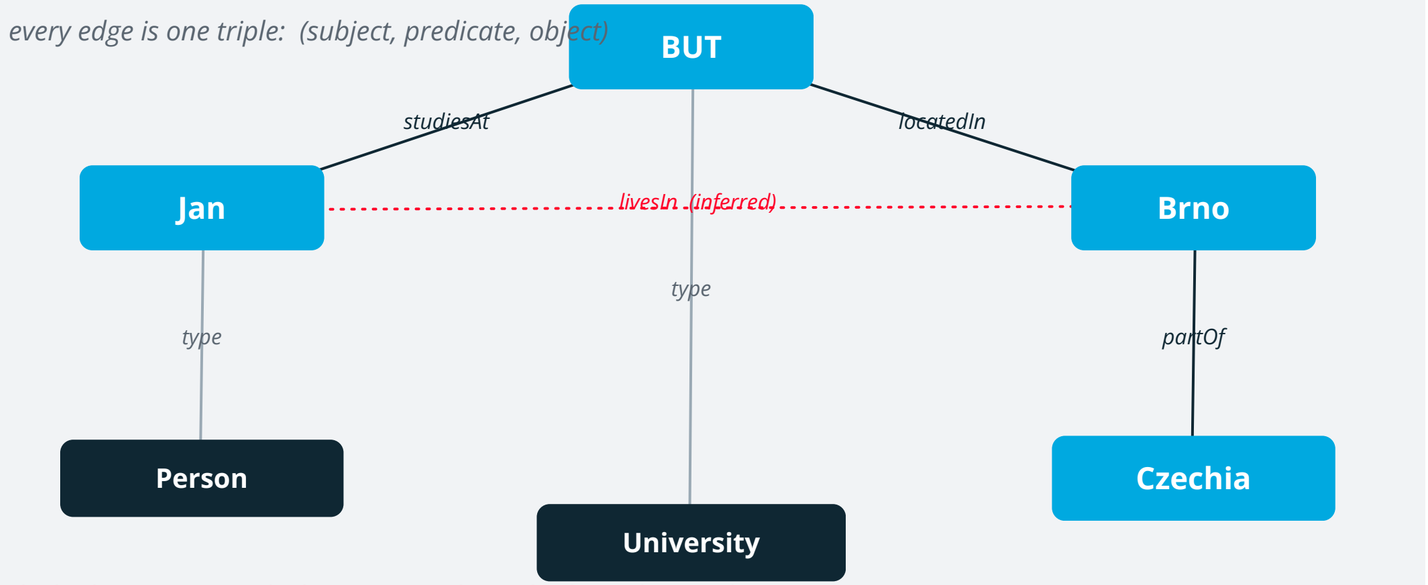
Preconditions	what must hold before the action is legal
Effects	exactly what changes, and what stops being true
The planner	searches for a sequence from the start state to the goal
The guarantee	if the model of the world is right, the plan is correct by construction
The catch	“if the model of the world is right”, and perception supplies that model

This is the robotics pipeline from the applications grid: a network labels the scene, the planner builds a plan, and the plan is verified before a motor turns.

A verified plan over a wrong world state is still wrong. It is just carefully executed.

Knowledge graphs and ontologies

Storing symbols at scale, and deriving new ones



The ontology

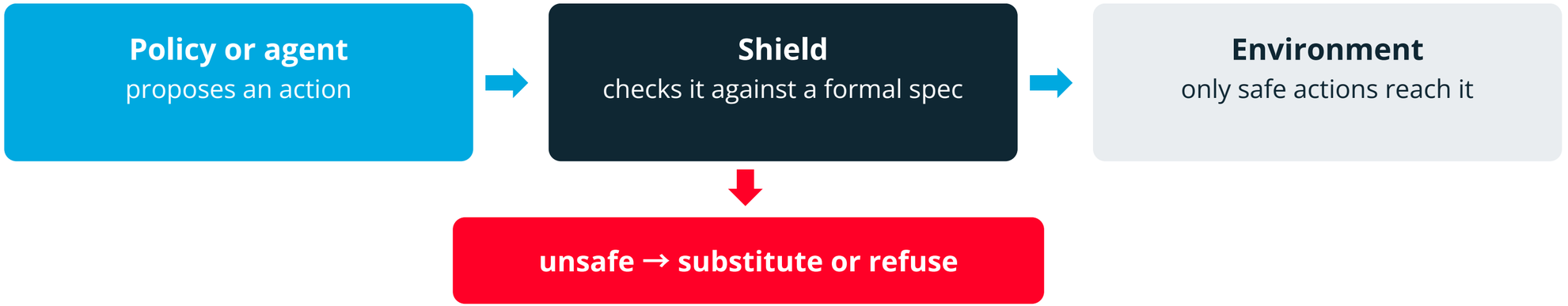
```
Person ⊑ Agent
University ⊑ Organisation

studiesAt ∘ locatedIn
        ⊑ livesIn

partOf is transitive

hasBirthDate is functional
```

Types and constraints, not data.

What the reasoner gives	entailed facts you never stored, plus detection of statements that contradict the schema
Description logics	the family behind OWL, kept weaker than first-order logic so reasoning stays decidable
Where you meet them	Wikidata, product and parts catalogues, biomedical ontologies, fraud and compliance graphs
The neurosymbolic link	a model extracts triples from text; the ontology decides which of them are admissible

Shields and runtime monitors

The pattern when the model is already deployed and you cannot retrain it



The idea	A safety property is written in temporal logic, then compiled into an automaton that runs alongside the agent.
The guarantee	The property holds on every step, whatever the model proposes. Alshiekh et al., AAAI 2018.
Why it is attractive	It needs no access to the model. You can shield a black box, or a vendor's API.
Where you meet it	Safe reinforcement learning, robotics, industrial control, and increasingly agent tool-use policies.
The limit	It can only enforce what someone wrote down. An unstated hazard is unshielded.

The “checked” level of the guarantee spectrum, and the one you can add to a system you did not build.

Is chain of thought neurosymbolic?

Two minutes. Argue before I answer.

The case for

- Intermediate steps become explicit text
- The problem is decomposed before it is answered
- Individual steps can be checked by something else
- With tool calls, some steps really are executed

The case against

- The steps are generated text, not verified inference
- The written reasoning need not be the real computation
- No schema, no types, no guarantee of any kind
- Nothing detects a wrong step except the next token

Chain of thought is a prompting strategy. It turns neurosymbolic the moment an external engine executes or checks a step.

Choosing an architecture

Five questions, in the order you should ask them

If …	then …
1 The stored facts settle the answer	**Rule engine or solver, deterministic path**
2 A constraint must never be violated	**Constrained decoding, or a hard post-check**
3 Input is perception-heavy or ambiguous	**Neural, grounded into a typed schema**
4 You need an audit trail	**Symbolic path; log the inputs and the rule that fired**
5 The domain changes every week	**Learn it. Rules you cannot maintain are worse than none**

Most real systems use three of these five rows at once, in different places. The architecture is a set of local choices, not one global one.

What structure buys you

Interpretability, reproducibility, verification

1 Traceability

The answer points at the facts it used and the rule that fired.

2 Reproducibility

A pure function returns the same output for the same input, every time.

3 Verification

Check the output against the constraints before it leaves the system.

4 Debuggability

Fixing a rule is a code change, not a training run.

5 Data efficiency

A rule encodes what would otherwise take thousands of examples.

Caveat

Greedy decoding removes sampling, not batching effects. A neural path is not bit-reproducible just because the temperature is zero.

For any system that claims to reason: which part is guaranteed, and by what?

03

In practice

Applications, limits, discussion

One question, end to end

Where it is already deployed

What the field has not solved

A worked example

One question, end to end

“Am I eligible for the reduced fare?”

1 **Neural: read the question and the record, extract typed facts**

```
age = 19,   status = student,   valid_id = true
```

2 **Symbolic: ordered rules, first match wins**

```
when age < 15              →  Child fare
when student and age<26    →  Student fare
when age >= 65             →  Senior fare
otherwise                  →  Full fare
```

3 **Decision**

Student fare, from rule 2, on age and status

4 **Neural: render it in plain language**

then check the rendered text still carries rule 2's answer

If no rule matches, the system does not improvise. It reports no match and takes the defined fallback.

A second worked example

Same pattern, different domain: a contract compliance check

“Does this supplier contract meet our procurement policy?”

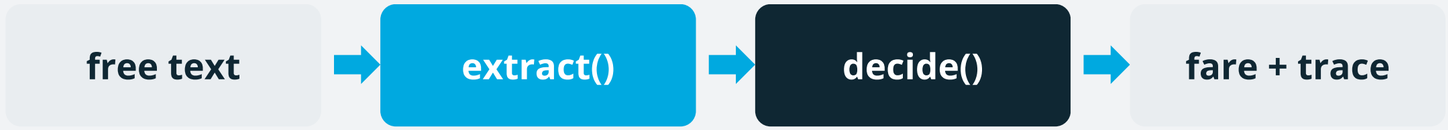
1	**Schema first**	clause_type, liability_cap, payment_days, governing_law, termination_notice
2	**Neural extraction**	a model reads 40 pages and fills those five fields, with a span citation for each
3	**Validation**	cap in EUR, payment_days an integer 0–180, law from a closed list, or abstain
4	**Rules**	cap ≥ 2× annual value; payment ≤ 60 days; law ∈ {CZ, DE, AT}; notice ≥ 90 days
5	**Output**	pass, fail, or escalate, naming the clause, the rule and the page

Note what did not change: five fields, ordered rules, a defined refusal, a citable trace. Only the nouns are different.

Optional lab 2: break the grounding

If the pace allows. The rules are already perfect.

You are given a working pipeline



Your rules score 10/10 on gold facts.
The system scores 5/12 on real text.

Seven wrong fares, each delivered with a complete, plausible, fully auditable derivation.

2a Find the confidently wrong one. Say in one sentence why the trace does not help you.

2b Make ground() abstain: validate against the schema, and refuse when the extractor guessed.

2c Zero confidently-wrong answers, at the lowest abstention rate you can reach.

Same ZIP as Task 1

`python3 check_task2.py`

Abstaining on everything scores zero wrong answers and is worth nothing.

Four ways it goes wrong

Only one of the four is caught automatically

was the delivered ANSWER right? \ were the extracted FACTS right?	yes	no
yes	**The happy path** Right facts, right answer, honest trace. This is what the demo shows.	**Right for the wrong reason** A 70-year-old jokes about being “a student of life” → student = true. The senior rule fires anyway. Fare correct, trace a lie. ***invisible to end-to-end tests***
no	**Rule bug** Facts were fine; the rule set was wrong or incomplete. The one kind your unit tests actually catch.	**Quiet failure** “I have 2 kids and I'm 34” → age = 2 → child fare. In range, correctly typed, completely wrong. ***caught by nothing***

and two outcomes that are not on the grid, because the system declined to answer

- **Loud failure** — age extracted as 1958: outside the schema range, so validation catches it and the system abstains
- **Correct refusal** — the passenger never states an age. Abstaining is the right answer, not a miss

Quick quiz

Two minutes in pairs, then we compare

1. **A language model writes an SQL query and the database executes it.**

   Who is in control, and what is guaranteed?

2. **A convolutional network labels objects, a PDDL planner builds the robot's plan.**

   Which stage can fail silently?

3. **A network is trained with a loss term that penalises constraint violations.**

   Does the constraint hold at inference time?

Name the Kautz type, then say what is actually guaranteed. The second half is the one that counts.

Where it is used

Anything with an audit requirement is a candidate

1 Mathematics and proof

AlphaGeometry; LLM-guided theorem proving in Lean

2 Software

Program synthesis, static analysis, models paired with SMT solvers

3 Healthcare

Clinical guidelines as rules over findings extracted by a model

4 Legal and compliance

Contract and policy checks with a citable derivation

5 Robotics

Perception feeds a PDDL planner; the plan is verified before execution

6 Industry

Diagnostics over knowledge graphs, configuration and pricing engines

Research: simulated populations of typed individuals for market and policy studies. A neural layer generates plausible respondents, a symbolic layer enforces that the population matches the real distribution.

The same shape recurs: a model reads the world, a rule decides what follows.

Where the commercial case comes from

Traceability is increasingly a regulatory requirement

EU AI Act	High-risk systems carry obligations on logging, technical documentation, transparency and human oversight. Phased application runs through 2026–27.
GDPR	Where a decision is automated and significant, people have rights around it, including meaningful information about the logic involved.
Sector rules	Finance, medical devices and safety-critical engineering had audit and validation requirements long before any of this.
What auditors want	Not a saliency map. The inputs, the rule that applied, the version of that rule, and the timestamp.
What that implies	A system whose decisions are computed by an inspectable procedure is far cheaper to certify than one whose decisions are sampled.

This is not a legal briefing. Check the current text before you rely on any of it.

The commercial argument for the symbolic half is usually written by a regulator, not an engineer.

Benchmarks in this field

What people measure on, and what each one is really testing

MNIST-Addition	Learn digits from sums alone	*Weak supervision through logic*	DeepProbLog, 2018
CLEVR	Questions about rendered 3-D scenes	*Compositional visual reasoning*	Johnson et al., 2017
CLUTRR	Kinship relations from short stories	*Systematic generalisation to longer chains*	Sinha et al., 2019
SCAN	Compositional command following	*Whether "jump twice" works after learning both parts*	Lake & Baroni, 2018
ProofWriter / RuleTaker	Reasoning over stated rules in text	*Multi-step entailment, not recall*	Clark et al., 2020
GSM8K	Grade-school word problems	*Arithmetic chains, a known neural weak spot*	Cobbe et al., 2021

Notice the pattern: almost all of them test generalisation to structures longer or deeper than anything in training.

How to read a paper in this field

Six questions, in this order

1 Where is the boundary?

Which component is learned, which is computed, and what crosses between them?

2 Soft or hard?

Is the constraint encouraged during training, or enforced at inference? Slide 18 is the checklist.

3 What is the baseline?

Compared against a plain neural model of the same size, or against a weaker one?

4 What is the train/test split?

If test structures are no deeper than training ones, no generalisation claim has been supported.

5 Does it scale past the toy?

MNIST-Addition is four lines of Prolog. Ask what happens at realistic scale.

6 Who wrote the knowledge?

If the rules were hand-authored for the benchmark, that cost belongs in the results.

Question two eliminates most of the overclaiming in this literature on its own.

Challenges and open problems

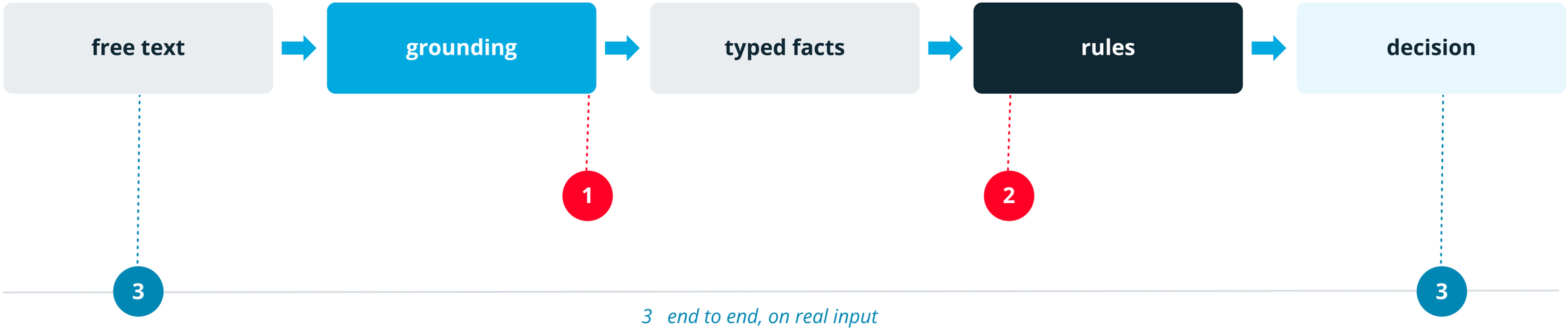
Known limitations

Grounding	Reliably turning messy input into clean symbols is still the hardest part.
Knowledge acquisition	Rules are expensive to write, review and keep current.
Coverage	What happens when nothing matches, and who notices.
Scale	Symbolic search is combinatorial; naive reasoning does not scale.
Evaluation	Few shared benchmarks, so competing claims are hard to compare.
Engineering	Two stacks, two skill sets, one interface that must stay in sync.
Overclaiming	“Deterministic” and “explainable” are used loosely. Ask what exactly is guaranteed.

In practice most failures are grounding failures, not reasoning failures.

How to evaluate one

Four measurements, and where each one is taken



3 end to end, on real input

1	**Grounding accuracy**	Extracted facts vs labelled gold facts.	Its own labelled set. Nothing else finds a wrong fact behind a right answer.
2	**Rule correctness**	The reasoner run on gold facts.	Unit tests, not a benchmark. Should be 100%, or the rules are wrong.
3	**End-to-end accuracy**	The full pipeline on real input.	The number stakeholders ask for, and the least diagnostic of the four.
4	**Abstention rate**	How often the system declines.	Meaningless alone. Read it against the confidently-wrong count.

92% end to end with 0% abstention is worse than 84% that refuses the other 16%. Which you prefer is a policy decision, not a metric.

Building one yourself

A starter stack for a student project

A recipe

1. Write the schema first: the typed facts your rules will read
2. Author ten rules by hand, cover the common cases, define the fallback
3. Add grounding: a model with structured output, validated against the schema
4. Log every decision: the facts used, the rule that fired, the output
5. Measure the two halves separately, and keep the rules under tests

Tools worth knowing

Types
JSON Schema, Pydantic

Rules
an ordered rule set, or SWI-Prolog

Solvers
Z3 (SMT), Clingo (ASP)

Planning
PDDL, Fast Downward

Extraction
structured outputs, constrained decoding

Start with the schema, not with the model. The schema is the contract everything else has to honour.

Key takeaways

1. Neural for perception and language, symbolic for whatever has to be exact
2. The interface between the two is the actual design problem
3. Architectures differ mainly in who is in control and where the guarantee lives
4. Interpretability comes from structure, not from a generated explanation
5. Most failures are grounding failures, and the dangerous ones are quiet
6. The cost is knowledge engineering, and it does not go away

Which part is learned, which is computed, and what is actually guaranteed?

Glossary

Reference sheet

Grounding	turning raw input into typed symbols
Entailment	“follows from”: if the premises hold, so must the conclusion
Backward chaining	from a goal, work back to the facts
Fallback	the defined outcome when no rule matches
t-norm	a smooth stand-in for AND on the interval [0,1]
Constrained decoding	masking illegal tokens so invalid output cannot occur
SMT	SAT plus theories: arithmetic, arrays, bit-vectors
Ontology / schema	which facts exist and what values they may take
Forward chaining	from facts, derive all consequences
Unification	matching a goal to a rule and binding its variables
Abstention	refusing to decide. An outcome, not an error.
Neural predicate	a logical predicate whose truth value a network supplies
Unsat core	the minimal set of constraints that conflict
Soft vs hard	encouraged during training vs enforced at inference

Test yourself

Six questions covering the session

Explain

Why can a system be perfectly traceable and still be wrong?

Classify

An LLM writes a SQL query and a database runs it. Kautz type? What is guaranteed?

Distinguish

Semantic loss and constrained decoding both “add logic”. What is the difference that matters?

Apply

You must never quote a price below cost. Where do you put that rule, and why there?

Evaluate

A system reports 92% accuracy. Name two numbers you would ask for before believing it.

Design

Sketch a neurosymbolic system for one task from your own field. Name the schema and the fallback.

The last one is the only one without a right answer, and it is the one worth doing.

Further reading

Start with two: Kautz for the map, AlphaGeometry for the ambition

Foundations

Kautz (2022)
The Third AI Summer, AI Magazine

Garcez & Lamb (2023)
Neurosymbolic AI: The 3rd Wave

Besold et al. (2017)
Neural-Symbolic Learning and Reasoning: A Survey

Marcus (2020)
The Next Decade in AI

Systems and methods

Manhaeve et al. (2018)
DeepProbLog, NeurIPS

Badreddine et al. (2022)
Logic Tensor Networks, Artificial Intelligence

Xu et al. (2018)
A Semantic Loss Function, ICML

Trinh et al. (2024)
Solving olympiad geometry without human demonstrations, Nature

Slides + lab

Available after the lecture

Questions afterwards: ipolisensky@fit.vut.cz

Questions?

Thank you

ipolisensky@fit.vut.cz